

PySpark — Easy Guide, Scenarios & Interview Prep

The Python API for Apache Spark, explained simply with real examples

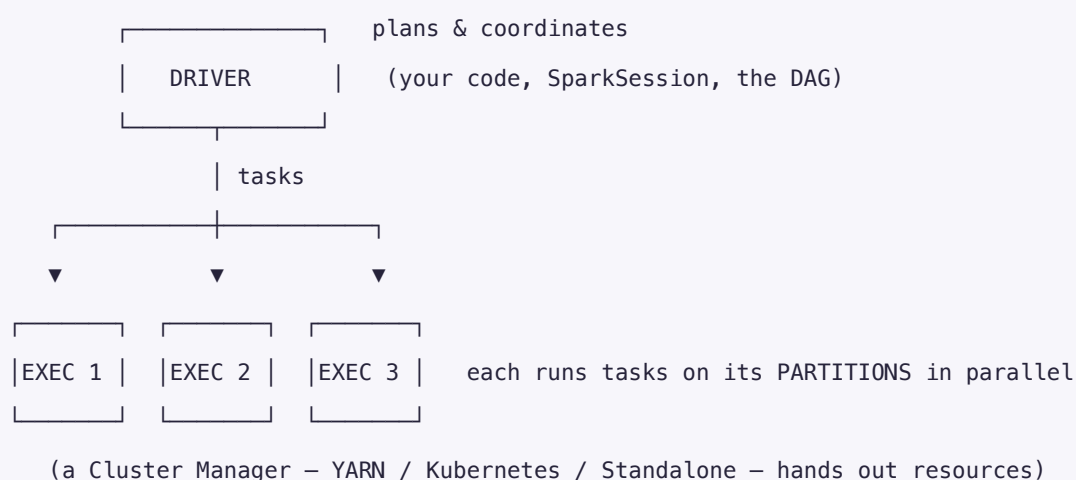
[Job execution flow](#) · [why/where/when-not](#) · [DataFrames](#) · [performance](#) · [errors](#) · [Q&A](#)

A practical walkthrough of PySpark (the Python API for Apache Spark), built around the topics in the SparkByExamples tutorial but explained simply, with runnable snippets, real scenarios, and the interview questions you'll actually get. Ties directly to your **Glue / Databricks** work. Running example: an **orders + customers** dataset at a retailer.

1. What PySpark is (and why it exists)

PySpark = Python API for Apache Spark, a distributed engine that processes data across a **cluster** of machines in parallel. When a dataset is too big (or too slow) for one machine or plain pandas, Spark splits it into **partitions** and processes them across many workers.

Architecture (know this for interviews):



- **Driver:** runs your program, builds the execution plan (a **DAG**), schedules work.
- **Executors:** the workers that actually process partitions and hold cached data.
- **Cluster manager:** allocates executors (in **AWS Glue** and **Databricks** this is managed for you — you just pick worker size/count).

In **Glue** you write this exact PySpark; Glue provisions the driver+executors. In **Databricks** you run it in notebooks on a cluster. Same API.

1B. Why, where, when NOT to use it, and common errors

Why we use PySpark

1. **Scale beyond one machine** — splits data into partitions across a cluster; handles GBs→PBs.
2. **Speed** — in-memory processing + parallelism across executors.
3. **One engine, many workloads** — batch ETL, SQL, streaming, and ML in one API.
4. **Auto-optimized** — Catalyst + lazy evaluation rewrite the whole plan (pushdown, pruning).
5. **Python + ecosystem** — familiar Python, integrates with S3/ADLS, Snowflake, Kafka, Delta.

6. **Fault tolerance** — DAG/lineage recomputes lost partitions if a node dies.

7. **Industry standard** — it's what **AWS Glue** and **Databricks** run under the hood.

Where it's used

Data-lake ETL/ELT (raw → partitioned Parquet), large joins/aggregations, CDC/incremental "latest record per key", streaming (Structured Streaming + Kafka/Kinesis), ML feature engineering/training-data prep, log/IoT processing, and large migrations/backfills.

Best-case scenarios (where it shines)

- Data is **too big for one machine** (won't fit in pandas).
- **Heavy transformations** — multi-table joins, wide aggregations, windows over large data.
- **Parallelizable, batch** (or micro-batch streaming) work.
- Reading/writing **columnar formats** (Parquet/ORC/Delta) from a lake.
- You need **one framework** across batch, SQL, streaming, and ML.

When NOT to use PySpark (know these — it signals maturity)

Situation	Why Spark is wrong	Use instead
Small data (< a few GB)	Startup + shuffle overhead makes it slower	pandas / DuckDB / SQL
Low-latency single-row lookups / OLTP	Batch, high-latency, not transactional	RDBMS, key-value store
Millisecond real-time per event	Structured Streaming is micro-batch	Flink (true streaming)
Simple SQL a warehouse can do	Wastes money doing what the warehouse does natively	Snowflake/Redshift/BigQuery
Many tiny files / many small jobs	Per-job overhead dominates	Batch them, or Lambda/pandas
Row-by-row / UDF-heavy logic	UDFs kill Catalyst; Python serialization is slow	Built-ins, or a different tool

One-liner: *"Spark is for big, parallel, batch workloads. For small data or low-latency single-record access it's overkill — shuffle and startup overhead make simpler tools faster and cheaper."*

Common PySpark exceptions/errors and how to fix them

Error / symptom	Root cause	Fix
<code>OutOfMemoryError</code> / executor OOM	Skew, huge shuffle, <code>collect()</code> to driver, low memory	Avoid <code>collect()</code> ; raise executor memory; repartition; filter early; fix skew
Job hangs on one task	Data skew (one key dominates)	Salt the key, enable AQE skew join , isolate the hot key
Everything slow / big <code>Exchange</code> in plan	Unnecessary shuffles (join/groupBy)	Broadcast small tables, filter before join, read <code>explain()</code>
Thousands of tiny output files	Too many partitions on write	<code>coalesce</code> / <code>repartition</code> before write; partition sensibly
<code>Task not serializable</code>	Non-serializable object (e.g., DB conn) in a transformation	Keep closures serializable; open connections in <code>mapPartitions</code>
<code>AnalysisException: cannot resolve column</code>	Wrong/ambiguous name or schema mismatch after join	Check <code>printSchema()</code> , alias/qualify columns
Slow/opaque UDF	UDF blocks Catalyst + serialization overhead	Use built-ins or a pandas (vectorized) UDF
<code>FileNotFoundException</code> / permission errors	Wrong S3/ADLS path or missing IAM/role	Verify path + credentials (Glue role / storage integration)
Driver OOM on <code>collect()</code> / <code>toPandas()</code>	Pulling a huge DataFrame to the driver	Write to storage or sample first
<code>ClassNotFound</code> / connector missing	Missing JAR (Kafka, Delta, JDBC)	Add the package/JAR to job/cluster config

Mental model: PySpark's cost is the **shuffle** and **cluster overhead** — a bargain when data is big and parallelizable, a waste when it's small or needs low-latency single-row access. Most production issues trace back to **skew, shuffles, memory, small files, or UDFs**.

1C. End-to-end: how a Spark job actually runs (high-volume walkthrough)

Scenario: RetailCo has **2 TB of order events/day** in S3 (Parquet) plus a tiny **regions** lookup. Goal: **revenue per region**. This is `filter → join → groupBy → write` , and the `groupBy` forces a **shuffle** — the interesting part.

The code

```
from pyspark.sql.functions import col, sum as _sum, broadcast
orders = spark.read.parquet("s3://retailco/curated/orders/dt=2026-08-10/") # 2 TB
regions = spark.read.parquet("s3://retailco/dim/regions/") # few MB
result = (orders
    .filter(col("status") == "completed") # narrow (no shuffle)
    .join(broadcast(regions), "region_id") # broadcast small table -> avoids big shuffle
    .groupBy("region_name") # WIDE -> shuffle
    .agg(_sum("amount").alias("revenue")))
result.write.mode("overwrite").parquet("s3://retailco/marts/revenue_by_region/") # ACTION
```

How it's submitted

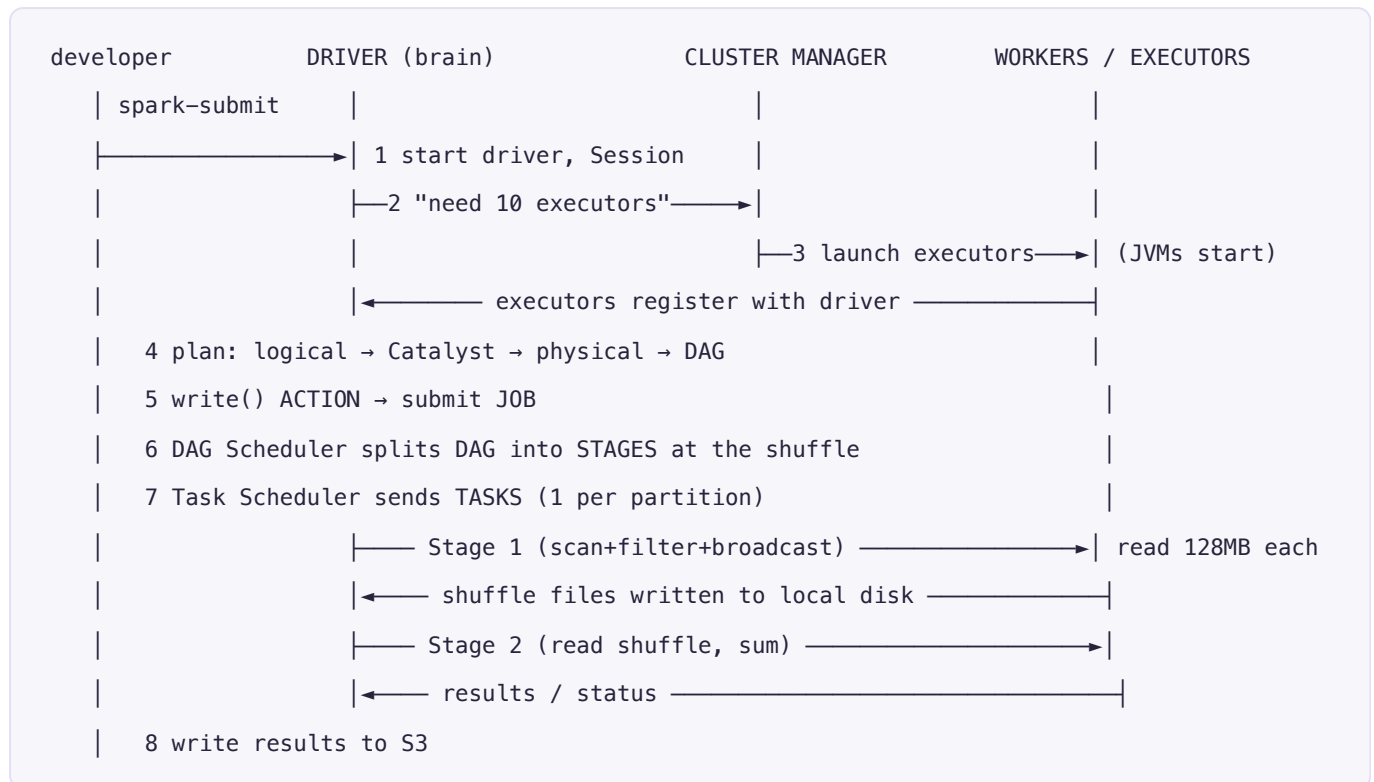
```
spark-submit --master yarn --deploy-mode cluster \
    --num-executors 10 --executor-cores 4 --executor-memory 16g revenue_by_region.py
```

(Glue/Databricks run this for you — you just pick worker size/count.)

The players

Component	Role
Driver	Runs your code + SparkSession, builds the DAG , schedules work. The brain.
Cluster Manager (YARN/K8s/Standalone)	Allocates resources; launches executors on workers.
Worker node	A machine hosting executors.
Executor	A JVM on a worker that runs tasks and holds cache/shuffle data. 10×4 cores = 40 parallel tasks .
Partition	~128 MB chunk = unit of parallelism. 2 TB ÷ 128 MB ≈ 16,000 partitions .
Task	Processes one partition on one core.
Job → Stage → Task	An action = a Job; split into Stages at shuffle boundaries; each Stage = many Tasks.

The flow



Step by step:

1. `spark-submit` asks the cluster manager to start the **driver**.
2. Driver creates the **SparkSession** (SparkContext inside).
3. Driver requests executors; the cluster manager **launches 10 executors** on workers; they **register back** with the driver.
4. As it reads your transformations (all **lazy**), the driver builds a **logical plan** → **Catalyst** optimizes (pushes the filter into the Parquet scan, prunes columns, picks a **BroadcastHashJoin**) → **physical plan** → **DAG**.
5. `write()` is an **action** → the driver submits a **Job**.
6. The **DAG Scheduler** splits it at the shuffle (`groupBy`) into **Stage 1** (scan → filter → broadcast-join, ~16,000 tasks) and **Stage 2** (read shuffle → sum, ~200 shuffle partitions).
7. The **Task Scheduler** ships **one task per partition** to executors. With 40 cores, ~40 run at once; 16,000 tasks flow through in ~400 waves.
8. **Stage 1 (narrow)**: each task reads its 128 MB partition, filters, tags region from the broadcast copy, and **writes shuffle files** to local disk (hashed by region).
9. **Shuffle**: data is **exchanged over the network** — each Stage-2 task pulls its region's pieces from every executor. (This is the `Exchange` in `explain()`.)
10. **Stage 2 (wide)**: each task sums `amount` per region.
11. Executors **write the small result** to S3. (If you'd used `collect()`, results return to the **driver** — fine here since it's tiny, dangerous for big outputs.)

Where Scala / the JVM fit

Spark's engine is **Scala on the JVM**; driver and executors are JVM processes. **PySpark is a thin Python layer** that talks to the JVM via **Py4J** — so **DataFrame/SQL ops run in the JVM at Scala speed** (Python only builds the plan). The exception is **Python UDFs**: rows are serialized from the JVM to a **Python worker** per executor and back, which is why UDFs are slow.

The numbers

2 TB ÷ 128 MB ≈ **16,000 partitions/tasks**; 10 executors × 4 cores = **40 parallel tasks** → ~400 waves; join is **broadcast** (no big shuffle); shuffle happens at `groupBy`; output is tiny.

What could go wrong here

- **Skew**: one big region → one Stage-2 task lags → AQE skew join / salting.
- **Forgot `broadcast()`**: the join would shuffle the full **2 TB** — minutes become hours.
- **Executor OOM / small files**: raise memory / add partitions; `coalesce` before writing.

2. SparkSession — your entry point

Every PySpark program starts by creating (or getting) a `SparkSession`:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("retail").getOrCreate()
```

It gives you `spark.read`, `spark.sql`, `spark.createDataFrame`, etc. (In Glue you get it via `GlueContext(SparkContext()).spark_session`.)

3. RDD vs DataFrame (what to use)

- **RDD (Resilient Distributed Dataset)**: the low-level, original Spark abstraction — a distributed collection you manipulate with `map`, `filter`, `reduce`. Powerful but no schema and no automatic optimization.
- **DataFrame**: a distributed table with **named, typed columns** (like a SQL table / pandas DataFrame). It runs through Spark's **Catalyst optimizer**, so it's faster and easier.

Interview answer: "Use DataFrames (or Spark SQL) almost always — they're optimized by Catalyst and easier to read. Drop to RDDs only for low-level control or unstructured data."

```
# RDD

rdd = spark.sparkContext.parallelize([1,2,3,4])

print(rdd.map(lambda x: x*2).collect())      # [2,4,6,8]


# DataFrame

df = spark.createDataFrame([(1,"Al"),(2,"Bo")], ["id","name"])

df.show()
```

4. Transformations vs Actions — lazy evaluation (a top interview topic)

- **Transformations** (`select`, `filter`, `withColumn`, `groupBy`, `join`) are **lazy** — they only build the plan; nothing runs yet.
- **Actions** (`show`, `count`, `collect`, `write`, `take`) **trigger execution** of the whole plan.

Why it matters: Spark sees the *whole* chain before running, so Catalyst can optimize it (push filters down, prune columns). "*Nothing executes until an action is called*" is a classic question.

[illegible]

5. DataFrame essentials (the daily 20%)

```
# read data (schema inferred; better to define it explicitly in prod)
orders = spark.read.option("header", True).option("inferSchema",
True).csv("s3://.../orders.csv")

orders.printSchema()          # column names + types
orders.show(5, truncate=False)

orders.columns                # list of column names
orders.count()                # row count

from pyspark.sql.functions import col, lit, when, upper, to_date

# select / rename / add / drop columns
orders.select("order_id", "amount").show()
orders.withColumnRenamed("amt", "amount")
orders.withColumn("amount_usd", col("amount") * 1.0)
orders.withColumn("order_date", to_date(col("created_at")))
orders.drop("temp_col")

# filter (WHERE)
orders.filter(col("status") == "completed")
orders.where("amount > 100 AND status = 'completed'")

# conditional column (CASE WHEN)
orders.withColumn("tier", when(col("amount") >= 1000, "high")
                        .when(col("amount") >= 100, "medium")
                        .otherwise("low"))

# sort, distinct, limit
orders.orderBy(col("amount").desc())
orders.select("status").distinct()
orders.limit(10)
```

Handling nulls & duplicates (real pipelines live here):

```
orders.na.drop()                # drop rows with any null
orders.na.fill({"amount": 0})   # fill nulls
orders.dropDuplicates(["order_id"]) # dedupe by key
```

6. Aggregations & groupBy

```
from pyspark.sql.functions import sum, avg, count, max, min, round

orders.groupBy("status").count().show()

orders.groupBy("customer_id").agg(
    count("*").alias("orders"),
    round(sum("amount"), 2).alias("total_spent"),
    avg("amount").alias("avg_order")
).orderBy(col("total_spent").desc()).show()
```

7. Joins (and the interview trap)

```
# join orders to customers
result = orders.join(customers, orders.customer_id == customers.id, "inner")

# join types: inner, left, right, outer, left_semi, left_anti
lost = orders.join(customers, "customer_id", "left_anti") # orders with no matching customer
```

Interview trap — the shuffle: a normal join **shuffles** data across the network (slow on big data). If one side is **small**, use a **broadcast join** to avoid the shuffle:

```
from pyspark.sql.functions import broadcast

orders.join(broadcast(customers), "customer_id") # sends small table to every executor
```

8. Window functions (ranking, running totals — asked constantly)

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, rank, dense_rank, lag, sum as _sum

w = Window.partitionBy("customer_id").orderBy(col("amount").desc())

# top order per customer
orders.withColumn("rn", row_number().over(w)).filter("rn = 1")

# running total of spend over time
w2 = Window.partitionBy("customer_id").orderBy("order_date") \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
orders.withColumn("running_total", _sum("amount").over(w2))

# previous order amount
orders.withColumn("prev_amount", lag("amount").over(
    Window.partitionBy("customer_id").orderBy("order_date")))
```

9. Spark SQL (same power, SQL syntax)

```
orders.createOrReplaceTempView("orders")
spark.sql("""
    SELECT status, COUNT(*) AS n, ROUND(SUM(amount),2) AS revenue
    FROM orders GROUP BY status ORDER BY revenue DESC
    """).show()
```

DataFrame API and SQL are interchangeable — both compile to the same optimized plan. Use whichever reads better.

10. UDFs (and why to avoid them when you can)

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

@udf(returnType=StringType())
def bucket(amount):
    return "high" if amount and amount >= 1000 else "low"

orders.withColumn("tier", bucket(col("amount")))
```

Interview point: prefer **built-in functions** over UDFs — UDFs are a black box to Catalyst (no optimization) and slower (Python serialization). Use **pandas UDFs (vectorized)** if you must. Reach for a UDF only when no built-in does the job.

11. Reading & writing data (+ partitioning)

```
# read
df = spark.read.parquet("s3://.../curated/orders/")
df = spark.read.option("multiLine", True).json("s3://.../raw/events/")

# write — Parquet, partitioned by date (fast, cheap downstream)
df.write.mode("overwrite").partitionBy("order_date").parquet("s3://.../curated/orders/")

# modes: overwrite, append, ignore, error
```

Why Parquet + partitioning: columnar + compressed, and partition **pruning** lets later queries skip irrelevant folders. This is the #1 performance lever (and your resume's story).

12. Performance & optimization (the part interviewers dig into)

- **Lazy + Catalyst + Tungsten:** Spark optimizes the whole DAG (predicate pushdown, column pruning) and generates efficient code. Write clear transformations; let it optimize.
- **Partitions:** the unit of parallelism. Too few = underused cluster; too many tiny ones = overhead. `repartition(n)` (full shuffle, can increase) vs `coalesce(n)` (no shuffle, only decrease — great before writing to reduce small files).
- **Shuffle** = data moved across the network (joins, `groupBy`, `distinct`, `orderBy`). It's the expensive operation — minimize it; filter early; broadcast small tables.
- **Broadcast join:** send a small table to all executors to skip the shuffle.
- **Caching:** `df.cache()` / `df.persist()` when you reuse a DataFrame multiple times; call `unpersist()` when done. Don't cache things you use once.

- **Data skew:** one key has way more rows → one task lags. Fixes: **salting** the key, enabling **AQE** skew handling, or filtering the hot key separately.
- **AQE (Adaptive Query Execution):** Spark 3+ re-optimizes at runtime (coalesces shuffle partitions, handles skew, switches join strategies). Usually on by default.
- **File sizing:** avoid the **small-files problem** — `coalesce` / `repartition` before writing; aim for ~128MB+ Parquet files.
- **Select only needed columns; filter early** — less data scanned and shuffled.

Read the plan: `df.explain(True)` shows the physical plan — look for `Exchange` (shuffle), `BroadcastHashJoin` vs `SortMergeJoin`, and scan pushdowns.

13. Real-world scenarios (tell these as experience)

S1 — Clean & standardize raw orders (Glue-style ETL). Read messy JSON/CSV → drop cancelled + bad amounts → cast types → add `order_date` → dedupe by `order_id` → write Parquet partitioned by date.

```
clean = (orders
    .filter((col("status") != "cancelled") & (col("amount") > 0))
    .withColumn("amount", col("amount").cast("double"))
    .withColumn("order_date", to_date("created_at"))
    .dropDuplicates(["order_id"]))
clean.write.mode("overwrite").partitionBy("order_date").parquet("s3://.../curated/orders/")
```

S2 — Revenue by region (join + aggregate). Join orders to customers, group by region, sum revenue. Broadcast the small `regions` table.

S3 — Top product per customer (window function). `row_number()` over `partitionBy(customer)` ordered by amount, keep `rn = 1`.

S4 — Incremental / dedup "latest record." `row_number()` partitioned by key ordered by `updated_at DESC`, keep `rn = 1` — the everyday CDC/upsert pattern (mirrors Glue bookmarks / dbt incremental).

S5 — Fix a slow job (performance). Job was shuffling a huge table against a small lookup and writing thousands of tiny files. Fix: **broadcast** the lookup, **filter early**, **coalesce** before writing, partition by date. Runtime dropped sharply. (*This is a great "how did you tune Spark" answer.*)

S6 — Skewed join. One customer had most of the rows → one task hung. Fix: enable **AQE skew join** and **salt** the hot key.

14. Interview Q&A (rapid, confident answers)

- **RDD vs DataFrame?** DataFrame = schema + Catalyst optimization + easier; RDD = low-level, use rarely.
 - **Transformation vs Action?** Transformations are lazy (build the plan); actions (`show/count/collect/write`) trigger execution.
 - **What is a shuffle and when does it happen?** Data moved across the network for `join`, `groupBy`, `distinct`, `orderBy`, `repartition` — the main perf cost.
 - **repartition vs coalesce?** `repartition(n)` full shuffle (up or down); `coalesce(n)` no shuffle, only reduce — use before writing to cut small files.
 - **Broadcast join?** Send a small table to all executors to avoid shuffling the big one.
 - **cache vs persist?** `cache` = persist in memory (default); `persist` lets you choose the storage level. Use when reusing a DataFrame; `unpersist` after.
 - **How do you handle data skew?** Salting, AQE skew handling, or isolate the hot key.
 - **Why avoid UDFs?** Opaque to Catalyst + Python serialization overhead; prefer built-ins or pandas UDFs.
 - **narrow vs wide transformation?** Narrow (`map`, `filter`) = no shuffle, data stays on the partition; wide (`groupBy`, `join`) = shuffle across partitions.
 - **What's a DAG?** The lineage of transformations Spark builds and optimizes before running; it's also how Spark recovers lost partitions (fault tolerance).
 - **How is PySpark different from pandas?** Pandas = single machine, in-memory, eager; PySpark = distributed across a cluster, lazy, scales to huge data.
 - **What is AQE?** Adaptive Query Execution — runtime re-optimization (shuffle partition coalescing, skew, join switch) in Spark 3+.
 - **How do you read a query plan?** `df.explain(True)` — look for `Exchange` (shuffle) and the join type.
-

15. One-page code cheat-sheet

```
# session
spark = SparkSession.builder.appName("app").getOrCreate()

# read / write
df = spark.read.option("header",True).csv("path"); df = spark.read.parquet("path")
df.write.mode("overwrite").partitionBy("dt").parquet("path")

# columns
df.select("a","b"); df.withColumn("c", col("a")+col("b")); df.drop("x")
df.withColumnRenamed("old","new"); df.filter(col("a")>10); df.where("a>10")

# nulls / dedupe
df.na.drop(); df.na.fill({"a":0}); df.dropDuplicates(["id"])

# aggregate
df.groupBy("k").agg(sum("v").alias("s"), count("*").alias("n"))

# join
a.join(b, "key", "left"); a.join(broadcast(b), "key")

# window
w = Window.partitionBy("k").orderBy(col("v").desc())
df.withColumn("rn", row_number().over(w)).filter("rn=1")

# sql
df.createOrReplaceTempView("t"); spark.sql("SELECT * FROM t WHERE v>0")

# perf
df.repartition(8); df.coalesce(1); df.cache(); df.explain(True)
```

Practice tip: you can run all of this free in a **Databricks Community Edition** notebook or locally with `pip install pyspark`. Doing 5–6 of the scenarios hands-on is the best way to be ready for a live coding round.